Estrutura de Dados II

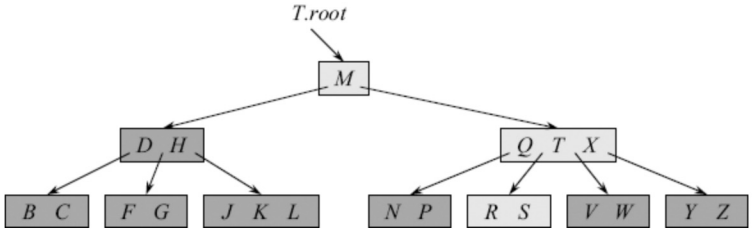
Prof. Vanessa Matias Leite

1

Árvores B

2

Árvores B



3

Árvores B

- São árvores balanceadas, desenvolvidas para otimizar o acesso a armazenamento secundário;
- São árvores de busca balanceadas projetadas para funcionar bem em discos ou outros dispositivos de armazenamento secundário de acesso direto;
- Devido à natureza dos discos rígidos, o acesso sequencial aos dados é muito mais eficiente que o acesso aleatório.
- Muitos sistemas de bancos de dados usam árvores B ou variantes de árvores B para armazenar informações.

4

Árvores B

- A organização das árvores de busca guia o acesso aos nós através dos ponteiros de árvore. Um nó por nível, no caso de uma busca.
- Como o acesso a disco é dispendioso, ao invés de transferir um dado de cada vez, busca-se transferir uma maior quantidade de informações a cada acesso.
- O tempo para acessar um byte em disco e um conjunto de bytes contíguos é praticamente o mesmo.

5

Árvores B

- A quantidade de dados utilizados numa árvore B obviamente não cabem na memória de uma só vez, por isso é necessário paginá-la;
- Especializações são feitas de acordo com as necessidades da aplicação. O fator de ramificação, por exemplo, pode variar de 3 a 2048 por exemplo (dependendo do buffer dos discos e do tamanho das páginas de memória alocados pelo SO).

6

Árvores B

- Árvores B podem ter muitos filhos, de alguns até milhares. Isto é, o “fator de ramificação” de uma árvore B pode ser bastante grande, embora normalmente, depende de características da unidade de disco utilizada.

7

Árvores B

- 6 milhões de componentes;
- 247 km de fiação;
- 8 km de tubulação;

8

Árvores B

- Inventor Rudolf Bayer;
- BAYER, Rudolf; MCCREIGHT, Edward. **Organization and maintenance of large ordered indices**. 1970. p. 107-141.

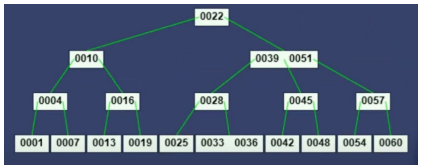
9

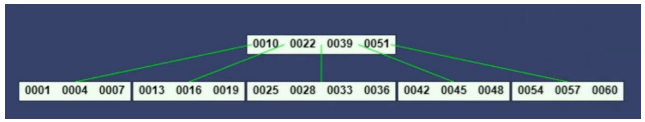
Propriedades Árvores B

- Toda árvore B deve ter um valor que representa a sua ordem;
- A ordem determina qual é número mínimo e o número máximo de chaves de um nó e consequentemente o número de filhos que cada nó da árvore pode ter.
- Quanto maior a ordem, mais achatada vai ser a árvore

10

Propriedades Árvores B





11

Propriedades Árvores B

Definição (interpretação de Alan Tharp)

Uma árvore B de ordem mínima t >= 1 é uma árvore com as seguintes propriedades:

- Cada nó possui os seguintes campos:

<NChaves, Folha, P_0 <K_1info$_1$>, P_1, <K_2info$_2$>, ..., <$K_q$info$_q$>, P_q>

- NChaves - número de chaves armazenadas no nodo;
- Folha - flag indicando se o nodo é folha ou não;
- P_i - ponteiro para o nodo filho i;
- K - chaves:
- Info$_i$ - ponteiro para o elemento de dados na memória ou para um registro em disco onde se encontra a chave K;

12

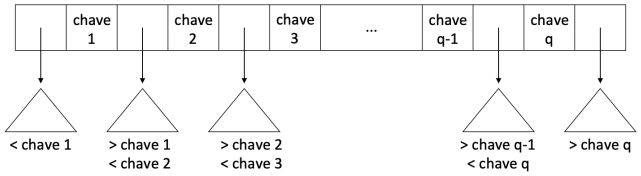
Propriedades Árvores B

Definição (interpretação de Alan Tharp)

- N° de chaves $t <= q <= 2t + 1$.
- A situação $q = 2t + 1$ é instável e provoca a explosão do nó no ajuste subseqüente da árvore.
- A situação $0 < q < t$ é estável somente na raiz. Quando é forçada por uma deleção fora da raiz, provoca a imediata rotação de uma chave ou fusão do nó com um irmão.

13

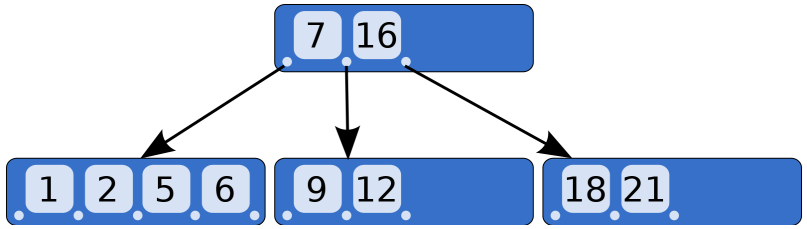
Propriedades Árvores B



14

Propriedades Árvores B

- Qual é a ordem desta árvore?



16

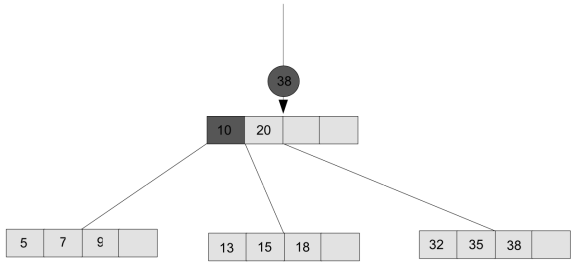
Busca de uma chave em Árvore B

1. pt = ponteiro p/ raiz da árvore

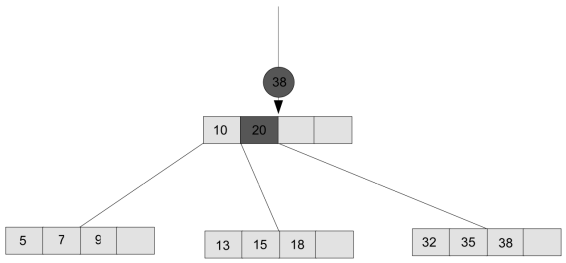
2. Percorra as chaves do nó apontado por pt, até encontrar X, ou até encontrar uma chave > X, ou até percorrer todas as chaves do nó

   a) Se encontrou chave X, encerre a busca retornando pt
   b) Senão, se encontrou chave maior que X, ptNovo = ponteiro da esquerda dessa chave
   c) Senão, se percorreu todas as chaves do nó atual, ptNovo = ponteiro da direita da última chave do nó
   d) Se ptNovo = NULL, encerre a busca, retornando pt (chave não está na árvore, mas, se estivesse, deveria estar no nó apontado por pt)
   e) Senão, pt = ptNovo, volte ao passo 2

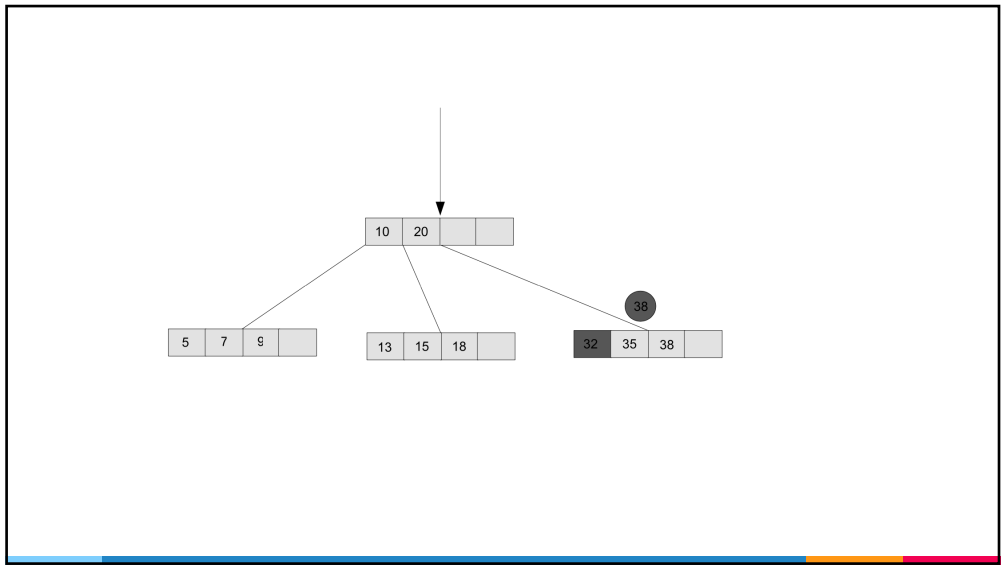
17



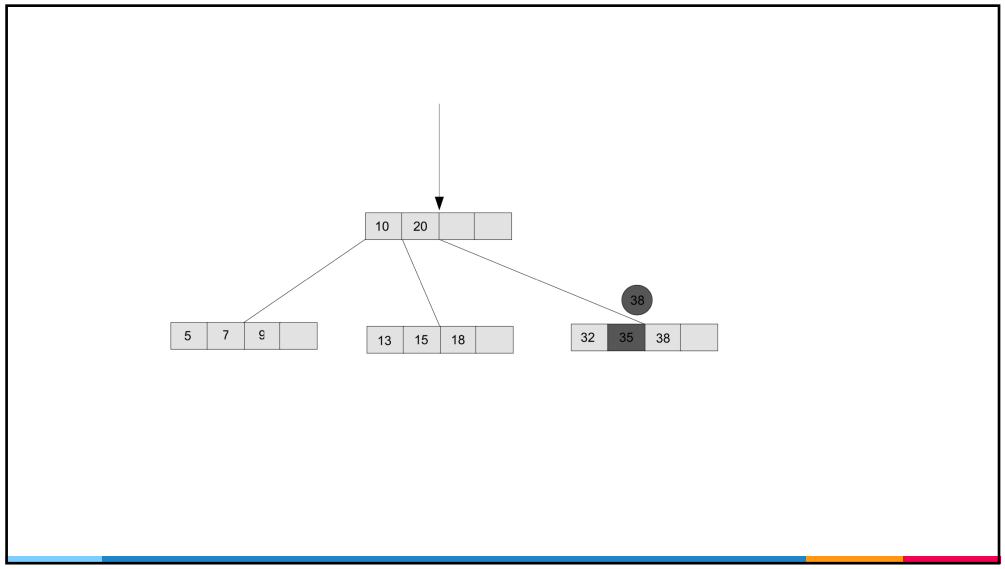
18



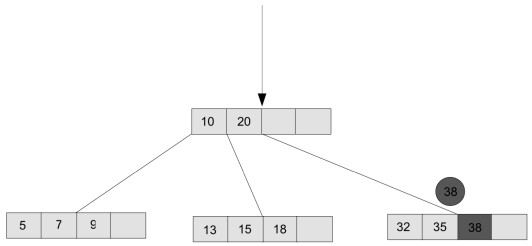
19



20



21



22

Inserção

a. Busca para encontrar a posição de inserção (**sempre em um nó folha**) ;
b. Se o nó estiver cheio, efetuar split do nó;
c. No caso de split, propagar a alteração para os nós acima;
d. A árvore cresce um nível na ocorrência de split da raiz;

23

Inserção

- desça a árvore em busca da chave;
  - a inserção é sempre feita nas folhas;
- ao retornar da inserção:
  - verifique se o nodo está cheio

    (se contém 2t+1 chaves);

  - se o nó estiver cheio:
    - dividir o nodo ao meio;
    - colocar a chave do meio no nó pai (se este também encher, repetir o processo recursivamente.)

24

Split

- Caso o nó esteja cheio, devemos separar (split) o nó ao redor do elemento mediano, criando 2 novos nós que não violam as definições da árvore;
- O elemento mediano é promovido, passando a fazer parte do nó pai daquele nó

25

Inserção com t=2

- 1° : 80, 50, 95, 90, 60
- 2°: 65, 70, 75
- 3°: 55, 64, 51
- 4°: 77, 78, 76
- 5°: 98, 99, 96

26

Remoção

- A remoção de uma chave é análoga à inserção, porém com alguns complicadores, já que uma chave pode ser removida de qualquer nó, seja ele folha ou não.
- Assim como na inserção, precisamos garantir que, ao removermos a chave as propriedades da árvore B não sejam violadas
- Da mesma maneira que tivemos de garantir que um nó não se tornasse grande demais na inserção, devemos garantir que ele não torne-se pequeno demais, ou seja, deve sempre ter pelo menos t − 1 elementos

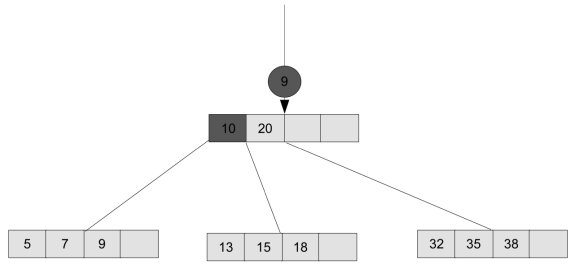
27

Remoção

Caso 1: Se a chave k estiver numa folha da árvore e a folha possui pelo menos t chaves, remove-se a chave da árvore;

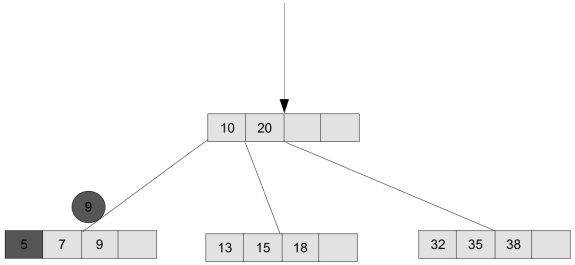
28

Remoção – Caso 1



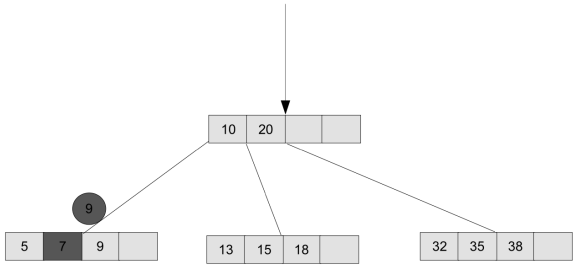
29

Remoção – Caso 1



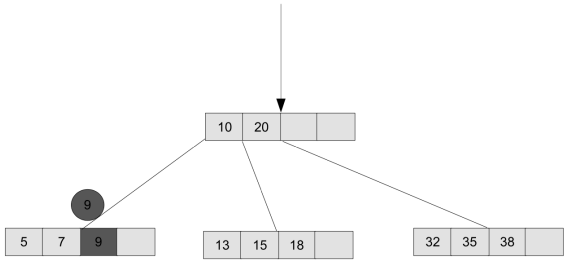
30

Remoção – Caso 1



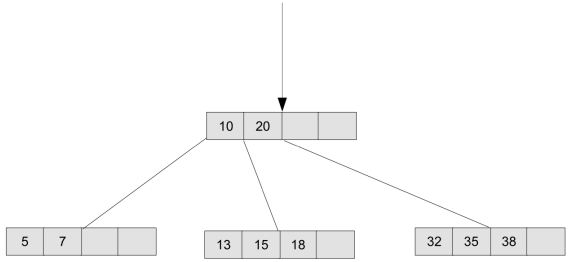
31

Remoção – Caso 1



32

Remoção – Caso 1



33

Remoção- Caso 2

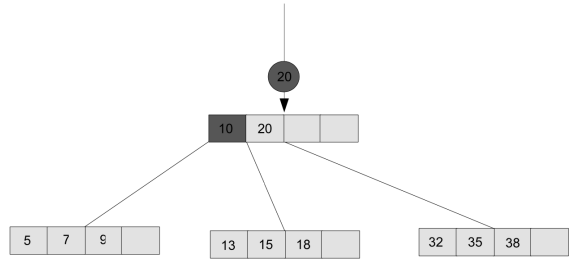
Se a chave k está em um nó interno x o seguinte deve ser feito:

a) Se o filho y que precede k no nó x possui pelo menos t chaves, encontre o predecessor k′ de k na sub-árvore com raiz em y. Remova k′ do nó filho e substitua k por k′ no nó atual;

b) Simetricamente, se o filho z que sucede k no nó x possui pelo menos t chaves, encontre o sucessor k′ de k na sub- árvore com raiz em z. Remova k′ do nó filho e substitua k por k′ no nó atual

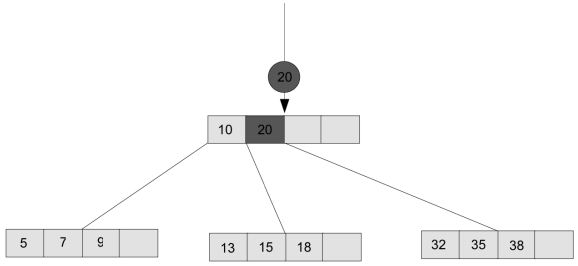
34

Remoção - Caso 2 (a/b)



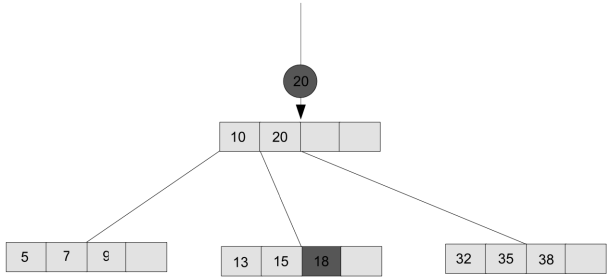
35

Remoção - Caso 2 (a/b)



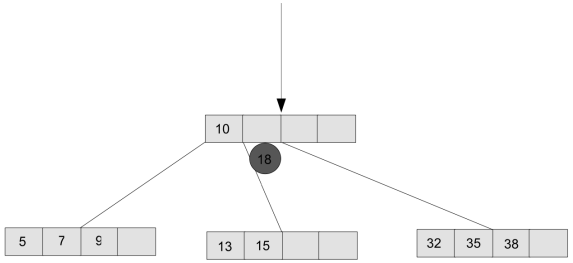
36

Remoção - Caso 2 (a/b)



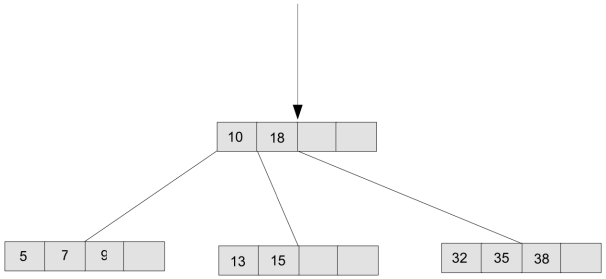
37

Remoção - Caso 2 (a/b)



38

Remoção - Caso 2 (a/b)



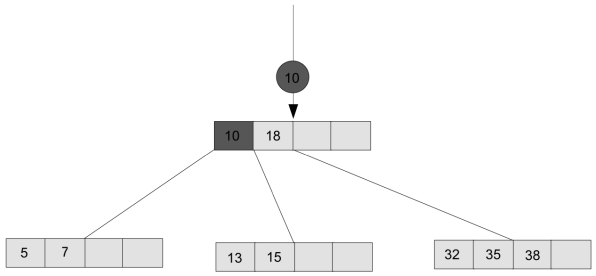
39

Remoção- Caso 2 (c)

c) Caso ambos y e z possuem somente t chaves, copie todos os elementos de z em y , libere a memória ocupada por z e remova o apontador em x e remova k de x.

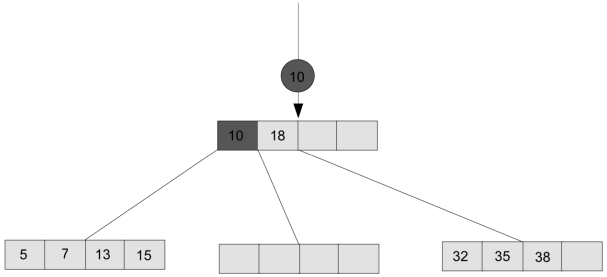
40

Remoção- Caso 2 (c)



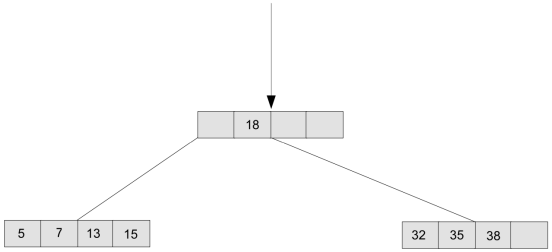
41

Remoção- Caso 2 (c)



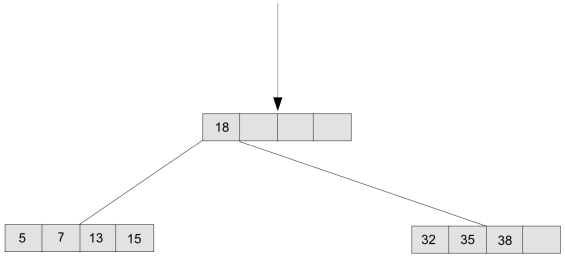
42

Remoção- Caso 2 (c)



43

Remoção- Caso 2 (c)



44

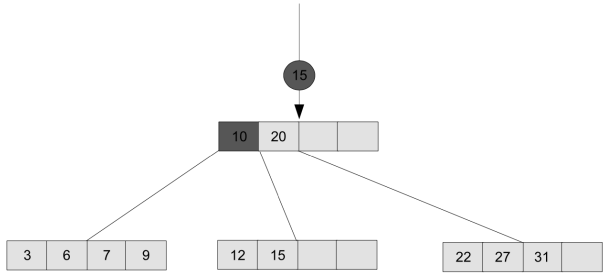
Remoção – Caso 3

Se a chave k não está presente no nó interno x, determine a sub-árvore $c_i[x]$ apropriada que deve conter k. Caso $c_i[x]$ possuir somente t chaves, proceder da seguinte maneira:

a) Se c_i [x] possui pelo menos t chaves mas possui um irmão adjascente com pelo menos t chaves copie para $c_i[x]$ uma chave extra, movendo uma chave de x para $c_i[x]$ em seguida movendo uma chave de um dos irmãos adjascentes a $c_i[x]$ de volta para x e ajustando o apontador para o nó correspondente

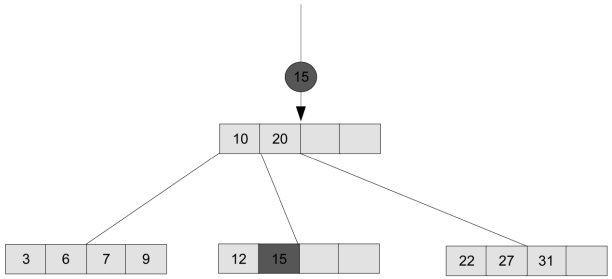
45

Remoção – Caso 3 (a)



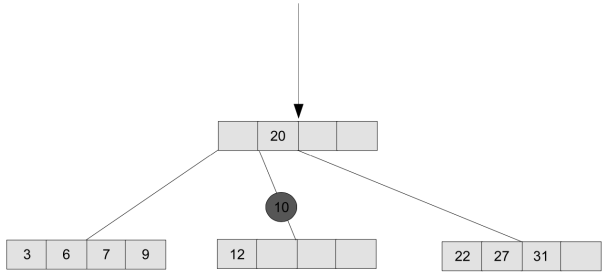
46

Remoção – Caso 3 (a)



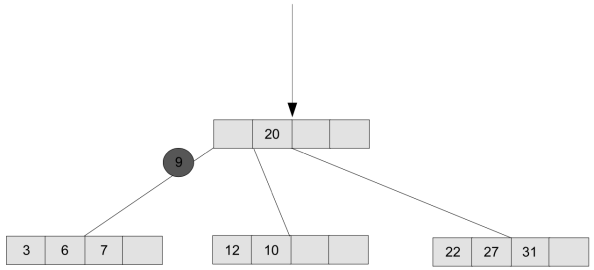
47

Remoção – Caso 3 (a)



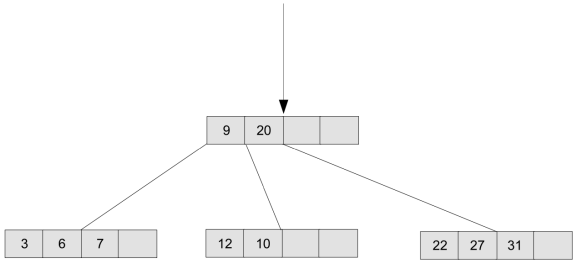
48

Remoção – Caso 3 (a)



49

Remoção – Caso 3 (a)



50

Remoção – Caso 3 (b)

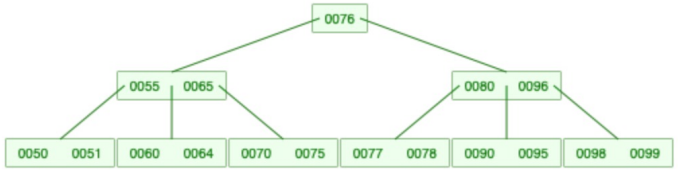
Se a chave k não está presente no nó interno x, determine a sub-árvore $c_i[x]$ apropriada que deve conter k. Caso c_i [x] possuir somente t chaves, proceder da seguinte maneira:

b) Se $c_i[x]$ e ambos os seus irmãos esquerdo e direito possuem t chaves, una c_i [x] com um dos irmãos o que envolve mover uma chave de x para o novo nó que acabou de ser formado, sendo que x é o elemento mediano daquele nó.

51

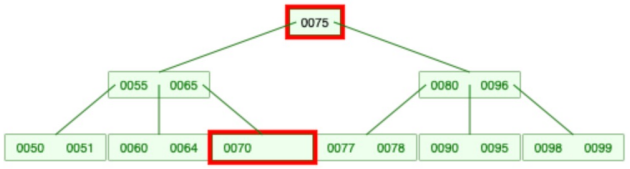
Remoção – Caso 3 (b)

- Remover o 76



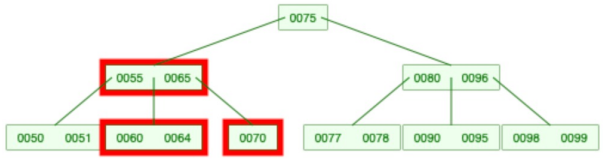
52

Remoção – Caso 3 (b)



53

Remoção – Caso 3 (b)

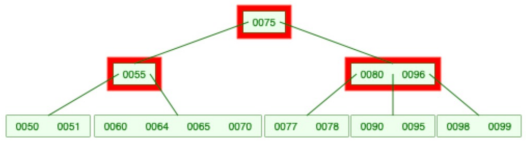


54

Remoção – Caso 3 (b)

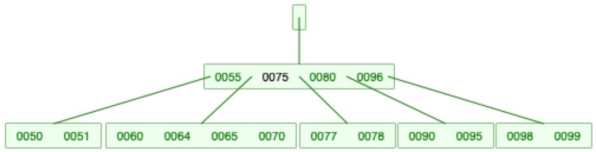
55

Remoção – Caso 3 (b)



56

Remoção – Caso 3 (b)



57

Remoção – Caso 3 (b)

58